

Geautomatiseerde Anonimisering en Pseudonimisering van Persoonsgegevens: Een Architectonische en Praktische Gids voor Lokaal Gehoste LLM-Infrastructuur in het Hoger Onderwijs

Inleiding tot Datasoevereiniteit en Privacy-Preserving AI in het Hoger Onderwijs

De exponentiële groei en integratie van artificiële intelligentie (AI), en in het bijzonder Grote Taalmodellen (Large Language Models, of LLMs), binnen het hoger onderwijs en wetenschappelijk onderzoek bieden ongekennde mogelijkheden. Deze technologieën faciliteren data-analyse op grote schaal, transformeren de onderwijsondersteuning door middel van gepersonaliseerde leeromgevingen, en optimaliseren administratieve processen. Echter, de verwerking van academische en administratieve data brengt aanzienlijke en complexe risico's met zich mee op het gebied van privacy en gegevensbescherming.¹ Binnen de Europese Unie stelt de Algemene Verordening Gegevensbescherming (AVG/GDPR) strikte, onwrikbare eisen aan de verwerking van Persoonsgegevens (Personally Identifiable Information, of PII).¹ Het routinematig gebruik van commerciële, cloud-gebaseerde AI-diensten—zoals OpenAI's ChatGPT of Anthropic's Claude—voor de verwerking van gevoelige studentgegevens, medische onderzoeksdata, HR-dossiers of ongestructureerde interviewtranscripties resulteert onvermijdelijk in data-exfiltratie.⁴ Deze praktijk levert niet alleen een potentieel datalek op, maar vormt ook een directe schending van de AVG wanneer er geen waterdichte verwerkersovereenkomsten en waarborgen voor datasoevereiniteit zijn geïmplementeerd.¹

Binnen de context van het Nederlandse hoger onderwijs en wetenschappelijk onderzoek benadrukken coöperatieve verenigingen zoals SURF met toenemende urgentie de noodzaak van datasoevereiniteit en het principe van "Privacy by Design".⁶ Innovatieve initiatieven zoals EduGenAI (een platform voor de verantwoorde ontwikkeling van generatieve AI in het tertiair onderwijs), de SURF AI-Hub, en de actieve ontwikkeling van het nationale GPT-NL taalmodel illustreren een fundamentele, sectorbrede beweging.⁷ Deze beweging is gericht op de realisatie van een onafhankelijke, lokaal gehoste IT-infrastructuur die de invloed van externe, voornamelijk Amerikaanse cloudleveranciers (vendor lock-in) drastisch minimaliseert.⁸ De relevantie van deze verschuiving wordt onderstreept door recente Data Protection Impact

Assessments (DPIA's), zoals de evaluatie van Microsoft 365 Copilot, waaruit onomstotelijk is gebleken dat ongecontroleerde datastromen naar gesloten, externe modellen significante en vaak onacceptabele privacyrisico's opleveren voor zowel medewerkers als studenten.¹¹ Het lokaal hosten van LLM-infrastructuur op interne servers—bij voorkeur on-premise uitgerust met krachtige GPU's voor adequate inferentiesnelheden—is derhalve niet slechts een technische voorkeur voor instellingen, maar een absolute juridische en ethische noodzaak.⁷

Het fundament van een dergelijke veilige, interne infrastructuur is de implementatie van geautomatiseerde, onfeilbare mechanismen voor anonimisering en pseudonimisering.¹² Dergelijke processen fungeren als een verplichte filterlaag (middleware) tussen de ongestructureerde brondata en het taalmodel, of tussen de eindgebruiker en de applicatie-interface. Dit rapport biedt een uitputtende, architectonische analyse van de theoretische kaders, de beschikbare open-source technologieën, en de praktische implementatie van PII-detectiesystemen. De nadruk ligt specifiek op oplossingen die naadloos integreren met lokale inferentie-engines zoals Ollama, die centraal beheerd kunnen worden via geavanceerde orkestratieplatforms zoals Open WebUI, en die fijnmazig geoptimaliseerd kunnen worden voor de Nederlandstalige, academische context. Het doel is om instellingen in het hoger onderwijs te voorzien van een direct inzetbare, praktische gids om een schaalbare en GDPR-inclusieve AI-omgeving te realiseren voor collega's, docenten en onderzoekers.

Fundamentele Concepten: Anonimisering, Pseudonimisering en Dataminimalisatie

De reductie van herleidbaarheid in datasets is een uiterst complex proces dat fundamenteel afhankelijk is van de beoogde toepassing van de resulterende data. Voordat men overgaat tot de selectie van software of modellen, is het cruciaal om het juridische en technische onderscheid tussen anonimiseren en pseudonimiseren te begrijpen, aangezien deze keuze de volledige technische architectuur van het AI-systeem dicteert.¹²

Anonimisering: Destructieve Databescherming

Anonimisering is per definitie een destructief proces waarbij persoonsgegevens en alle potentieel herleidbare attributen onomkeerbaar worden verwijderd of fundamenteel worden gewijzigd, zodanig dat de betrokkene met geen enkele beschikbare techniek meer identificeerbaar is.¹² In de context van tekstverwerking, log-analyse of document-parsing wordt dit vaak bereikt door middel van 'redactie' (het volledig weglaten of zwartlakken van de entiteit) of door onomkeerbare vervanging door een generieke placeholder (bijvoorbeeld de transformatie van "Student Jan Jansen woont in Amsterdam" naar "Student woont in").¹² Hoewel deze methode de hoogst mogelijke mate van privacybescherming biedt en de verwerkte data in veel gevallen buiten de scope van de AVG plaatst, introduceert het een significante daling in de semantische utiliteit van de tekst. De temporele, causale of relationele context gaat vaak onherroepelijk verloren.¹² Voor statistische aggregatie is dit acceptabel, maar voor complexe tekstanalyse door taalmodellen is dit problematisch.

Pseudonimisering en Bidirectionele Mapping

Pseudonimisering betreft de systematische vervanging van identificerende gegevens door een consistent pseudoniem, waarbij de originele data en de cryptografische sleutel of mapping-tabel om het pseudoniem om te keren, uiterst veilig en fysiek of logisch gescheiden worden bewaard.¹² Binnen de architectuur van interactieve AI-systemen, zoals LLM-gebaseerde chat-assistenten of onderzoeksanalysetools, is pseudonimisering vaak een absolute technologische vereiste. Wanneer een taalmodel een tekst analyseert om relaties te destilleren of een samenvatting te genereren, moet het de capaciteit behouden om onderscheid te maken tussen verschillende entiteiten. Als alle namen in een onderzoekstranscriptie worden geredigeerd naar een homogene [NAME] tag, verliest het LLM de mogelijkheid om te begrijpen wie wat zei, of welke spreker aan welke actie was verbonden.

Door het toepassen van consistente mapping—bijvoorbeeld "Jan Jansen" wordt systematisch vervangen door en "Piet Pieters" wordt—behoudt de tekst zijn complexe semantische en syntactische structuur, terwijl de identiteiten verborgen blijven voor het taalmodel.¹² Na de inferentiefase, wanneer het LLM een geanonimiseerde respons heeft gegenereerd, kan de lokaal opgeslagen mapping-tabel worden geraadpleegd om de originele namen weer in de output te substitueren (een proces dat bekend staat als deanonymization of de-identificatie).¹² Dit levert een naadloze, frictieloze gebruikerservaring op, zonder dat het onderliggende LLM, of de eventuele logfiles van dat model, de daadwerkelijke persoonsgegevens ooit hebben verwerkt of opgeslagen.

Impact van De-identificatie op Modelprestaties

Wetenschappelijk onderzoek naar de utiliteit van gesaneerde tekst voor machine learning benadrukt de noodzaak van zorgvuldige pseudonimisering. Studies naar de training van domeinspecifieke modellen (zoals clinical BERT-modellen op elektronische patiëntendossiers) tonen aan dat slechte anonimiseringstechnieken leiden tot een aanzienlijke verslechtering van de prestaties op stroomafwaartse taken.¹⁵ Een studie door Yeniterzi et al. illustreerde dat wanneer NER-modellen (Named Entity Recognition) werden getraind op inferieur gepseudonimiseerde data en vervolgens geëvalueerd op ongealtereerde tekst, de F1-score drastisch daalde van 0.862 naar 0.728.¹⁵ Lothritz et al. onderzochten de impact van de-identificatie op diverse datasets en concludeerden eveneens dat geavanceerde pseudonimiseringsstrategieën absoluut noodzakelijk zijn om de schade aan de bruikbaarheid van de data voor daaropvolgende AI-modellen te minimaliseren.¹⁵ Bovendien wees onderzoek van Berg et al. uit dat wanneer traditionele algoritmen (zoals Conditional Random Fields of LSTM-netwerken) worden getraind op gesaneerde tekst, de recall voor het detecteren van nieuwe PII aanzienlijk kan dalen (bijvoorbeeld van 85% naar 50%).¹⁵ Deze academische bevindingen onderstrepen het argument dat voor de inrichting van AI in het hoger onderwijs, het lokaal implementeren van een robuuste, hoogwaardige pseudonimiseringspijplijn essentieel is om de analytische kracht van de ingezette taalmodellen te behouden.

Evolutie van PII-Detectiemechanismen in Tekst

De daadwerkelijke detectie van gevoelige entiteiten, die logischerwijs voorafgaat aan de transformatie, vereist geavanceerde methodologieën. De technologie hiervoor heeft een aanzienlijke evolutie doorgemaakt, resulterend in verschillende klassen van oplossingen die vandaag de dag beschikbaar zijn.

1. **Reguliere Expressies (Regex) en Patroonherkenning:** Historisch gezien vertrouwden systemen vrijwel exclusief op regex en gedetailleerde, deterministische regels.² Regex is computationeel extreem goedkoop en uiterst efficiënt voor de detectie van sterk gestructureerde data, zoals e-mailadressen, creditcardnummers of IP-adressen.¹⁸ Het faalt echter catastrofaal bij ongestructureerde, contextafhankelijke data zoals namen van personen, organisaties of locaties, en is zeer vatbaar voor typefouten of afwijkende formattering.²
2. **Klassieke Named Entity Recognition (NER) via NLP:** Om de tekortkomingen van regex te ondervangen, verschoof de industrie naar machine learning en NLP-kaders zoals spaCy of Flair.¹² Deze systemen gebruiken getrainde statistische modellen om entiteiten te classificeren op basis van zinsbouw, woordsoorten en lexicale kenmerken. Hoewel ze veel beter omgaan met namen en locaties, zijn ze gelimiteerd door hun pre-gedefinieerde taxonomie (ze herkennen alleen de specifieke categorieën waarop ze zijn getraind) en vereisen ze aanzienlijke hoeveelheden gelabelde data om nieuwe entiteiten (zoals een uniek format voor een studentnummer) te leren herkennen.¹⁴
3. **Zero-Shot Modellen (GLiNER):** Recente doorbraken hebben geleid tot modellen zoals GLiNER (Generalist and Lightweight Named Entity Recognition), die bidirectionele transformator-architecturen gebruiken om willekeurige, door de gebruiker opgegeven entiteitstypes in tekst te vinden, zonder specifieke hertraining (zero-shot).²¹
4. **Generatieve AI (SLMs en LLMs):** De meest geavanceerde methode zet lokale, compacte Small Language Models (SLMs) of grotere LLMs in. Via zorgvuldig samengestelde prompts wordt het model geïnstrueerd om alle contextueel gevoelige informatie in een tekst te identificeren.² Waar regelgebaseerde engines vaak falen op ongestructureerde, 'rommelige' data zoals serverlogs of informele chatgesprekken, excelleren LLMs dankzij hun diepe semantische begrip.² De trade-off ligt hier in de computationele zwaarte en inferentietijd (latency) in vergelijking met regex.²¹

De architecturale uitdaging voor een onderwijsinstelling ligt in het vinden van de optimale synthese tussen deze methoden, balancerend tussen detectienauwkeurigheid, verwerkingssnelheid en de beschikbare GPU-rekenkracht op de lokale servers.

Uitputtende Analyse van Open-Source PII-Frameworks

Voor de inrichting van een robuuste, lokaal gehoste PII-infrastructuur moeten diverse

toonaangevende open-source instrumenten worden geëvalueerd en strategisch worden gepositioneerd. Deze systemen variëren van modulaire, op regels gebaseerde kaders tot gespecialiseerde neurale-gebaseerde extractiemodellen.

Microsoft Presidio: Het Industriestandaard Framework

Microsoft Presidio, waarvan de naam is afgeleid van het Latijnse woord "praesidium" (wat bescherming of garnizoen betekent), wordt algemeen beschouwd als een van de meest volwassen, uitgebreide en betrouwbare open-source kaders voor de detectie en anonimisering van gevoelige data in tekst en afbeeldingen.²³ Het is specifiek ontworpen om organisaties in staat te stellen privacy te behouden door middel van democratisering van de-identificatietechnologieën en ongeëvenaarde traceerbaarheid van de besluitvorming.¹⁴

De architectuur van Presidio is bewust opgesplitst in twee onafhankelijke microservices: de AnalyzerEngine en de AnonymizerEngine.¹⁷ Deze scheiding van verantwoordelijkheden (Separation of Concerns) is van immens belang voor de schaalbaarheid en auditbaarheid van het systeem.¹⁷ Het stelt data-engineers in staat om de detectie-logica te optimaliseren zonder het onderliggende anonimiseringsbeleid te wijzigen, of vice versa.¹⁷

De Analyzer Engine en Recognizer Registry

De AnalyzerEngine vormt het kloppend hart van de detectiefase en is een Python-gebaseerde service.²⁵ Het systeem maakt gebruik van een RecognizerRegistry, een uitgebreide verzameling van verschillende PII-herkenningsmodules (Recognizers) die gelijktijdig of sequentieel op de tekst worden losgelaten.²⁵ Presidio ondersteunt standaard een brede waaier aan globale entiteiten, waaronder CREDIT_CARD, CRYPTO (bijv. Bitcoin adressen), DATE_TIME, EMAIL_ADDRESS, IBAN_CODE, IP_ADDRESS, LOCATION, PERSON en PHONE_NUMBER.¹⁹

De engine combineert meerdere detectiestrategieën.²³ Voor ongestructureerde entiteiten zoals namen van personen (PERSON) en locaties (LOCATION) maakt Presidio in zijn standaardconfiguratie gebruik van spaCy als de onderliggende NLP-engine, leunend op pre-getrainde NER-modellen.¹⁴ Voor gestructureerde data implementeert het systeem PatternRecognizer objecten die reguliere expressies combineren met contextuele analyses en cryptografische validatielogica.²⁵

Een essentieel inzicht in de precisie van Presidio is het mechanisme van de score_context_improvement.¹⁷ Reguliere expressies leveren van nature een aanzienlijke hoeveelheid fout-positieven op. Presidio mitigeert dit probleem door de tekstuele context rondom een patroon te analyseren. Wanneer het systeem een willekeurige reeks van tien tot negentien cijfers detecteert via een regex, wordt hieraan in eerste instantie een relatief lage base confidence score (original_score) toegekend.¹⁷ Echter, de lemma-analysator scant de nabije omgeving op specifieke supportive_context_word tokens. Als woorden zoals "credit", "card" of "visa" zich binnen een specifieke afstand bevinden, past Presidio de score_context_improvement toe, waardoor de uiteindelijke betrouwbaarheidsscore drastisch stijgt.¹⁷ Bovendien voeren veel herkenkers na een patroon-match een validatie uit (zoals een

Luhn-checksum voor creditcards).¹⁷ Als deze validation_result faalt, kan de detectie worden verworpen, wat de robuustheid voor productieomgevingen garandeert.¹⁷

De Anonymizer Engine

Na detectie genereert de Analyzer een gestructureerde lijst met RecognizerResult objecten, die de start- en eindindex van de entiteit binnen de tekst, het type PII, en de berekende betrouwbaarheidsscore bevatten.¹⁷ Deze payload wordt vervolgens overgedragen aan de AnonymizerEngine.¹⁴ Deze engine past deterministische transformatie-operatoren toe op basis van vooraf gedefinieerde, configureerbare beleidsregels.¹⁴ Afhankelijk van het specifieke privacybeleid van de onderwijsinstelling, kan het systeem worden geconfigureerd om een e-mailadres te redigeren (vervangen door <EMAIL>), een telefoonnummer gedeeltelijk te maskeren (waarbij alleen de laatste vier cijfers zichtbaar blijven, e.g. *****5555), te hashen, of omkeerbaar te versleutelen met een cryptografische sleutel.¹⁷

Noodzaak voor Nederlandse Lokalisatie

Hoewel Presidio architectonisch superieur is, is het raamwerk in zijn standaardconfiguratie primair getraind en gericht op de Engelstalige en Noord-Amerikaanse context.²⁹ Voor effectieve inzet binnen Nederlandse instellingen voor hoger onderwijs vereist het framework fundamentele lokalisatie.²⁰ Dit omvat de integratie van Nederlandstalige spaCy-modellen of geavanceerde Flair-taalmodellen (flair/ner-dutch-large).²⁰ Daarnaast is de ontwikkeling van specifieke, aangepaste herkenner (Custom Recognizers) voor lokaal relevante entiteiten zoals het Burgerservicenummer (BSN), Nederlandse postcodes, en de formattering van Nederlandse vaste en mobiele telefoonnummers onontbeerlijk.¹⁴ Dit proces van uitbreiding wordt uitgebreid behandeld in de implementatiegids van dit rapport.

GLiNER: Zero-Shot Entity Recognition

Een significante tekortkoming van traditionele NER-systemen, zoals de standaard pipeline van spaCy of de basisimplementatie van Presidio, is hun rigide, pre-gedefinieerde taxonomie. Ze zijn exclusief getraind om een vastgestelde set van macro-entiteiten te herkennen (zoals PERSON, ORG, LOC). Het toevoegen van een geheel nieuw PII-type—bijvoorbeeld "Studentnummer" of "Faculteitsnaam"—vereist traditioneel het moeizaam verzamelen van grote hoeveelheden geannoteerde trainingsdata, gevolgd door een tijdsintensief en computationeel zwaar proces van fine-tuning van het model.¹⁴

Hier biedt GLiNER (Generalist and Lightweight Named Entity Recognition) een absolute paradigmaverschuiving. GLiNER is gebouwd op een bidirectionele transformatorarchitectuur en stelt systemen in staat tot "zero-shot" entiteitsextractie.²¹ In tegenstelling tot het voorspellen vanuit een gefixeerde lijst van klassen, accepteert GLiNER op het moment van inferentie zowel de ongestructureerde tekst als een dynamisch op te geven lijst van gewenste entiteitstypes.²¹ Het model identificeert autonoom de tekstfragmenten die semantisch overeenkomen met de zojuist opgegeven categorieën. Dit betekent dat een ontwikkelaar op runtime kan zoeken naar

`` zonder het model te hoeven hertrainen.²¹

Voor enterprise-grade privacy-toepassingen in onderwijs en onderzoek is met name het knowledgator/gliner-pii-base-v1.0 model (vaak in samenwerking met Wordcab) van exceptionele waarde.²¹ Dit specifieke model is uitgebreid gefinetuned voor de detectie van meer dan 60 gedetailleerde categorieën van PII, PHI (Protected Health Information) en PCI (Payment Card Industry) data.²¹ Onafhankelijke evaluaties, zoals de tests uitgevoerd op multi-domein PII-datasets, tonen aan dat geoptimaliseerde GLiNER-modellen een indrukwekkende F1-score van om en nabij de 81% behalen.³³ GLiNER presteert hiermee significant beter dan generieke spaCy-modellen of rigide regex-benaderingen, voornamelijk dankzij de superieure recall-ratio; het vangt met gemak onregelmatig geformatteerde ID-nummers en data op die generieke modellen over het hoofd zien, dankzij het diepe semantische begrip verworven tijdens training op synthetische data.³³

Cruciaal voor lokale implementatie op een GPU-server is dat GLiNER geoptimaliseerd kan worden middels quantisatie-technieken (zoals FP16 en UINT8 ONNX-modellen).²¹ Hierdoor is het extreem snel en lichtgewicht—veel sneller dan inferentie via enorme Large Language Models—waardoor het lokaal kan draaien zonder de pipeline te vertragen of buitensporig veel VRAM te consumeren.²¹ Het python-script voor integratie is elegant, zoals gedemonstreerd in document-parsing frameworks zoals Docling, waarbij GLiNER eenvoudig kan worden aangeroepen om entiteiten te extraheren voor obfuscation.³⁵

LLM-Gebaseerde Oplossingen: AnonDocs, PII-Redact en DataFog

De meest flexibele categorie van PII-detectie leunt volledig op de inzet van Large Language Models of Small Language Models. Bibliotheken zoals PII-Redact en DataFog, evenals platformen zoals AnonDocs en PII Guard, maken gebruik van de krachtige contextuele intelligentie van LLMs.³ Waar regelgebaseerde engines vastlopen op zwaar geobfusceerde of ongestructureerde data (zoals informele serverlogs vol met rommelige tekens, of lange, ongestructureerde chatgesprekken van studenten), kunnen LLMs op basis van hun brede taalkundige kennis de onderliggende gevoelige informatie nog steeds feilloos identificeren.²

AnonDocs is ontworpen als een opzichzelfstaande, privacy-first API voor het massaal anonimiseren van documenten (PDF, DOCX, TXT).³⁷ De architectonische filosofie van AnonDocs is "Zero Data Leakage" en "Zero Database"; het is een stateless dienst die bestanden in-memory verwerkt en na de transformatie direct weer vernietigt.³⁷ Wat AnonDocs uiterst relevant maakt voor lokale infrastructuur, is de diepe, native integratie met open-source LLM-providers, in het bijzonder Ollama.³⁷ In plaats van te leunen op traditionele NLP, stuurt AnonDocs de geëxtraheerde tekst (in parallelle chunks om de doorvoersnelheid te verhogen) naar een lokaal draaiend taalmodel, voorzien van strikte system prompts om entiteiten te identificeren en te redigeren.³⁷

OpenPipe's PII-Redact is een vergelijkbare open-source Python bibliotheek die speciaal profiteert van LLMs, zoals het compacte Llama 3.2 1B model, om PII in ongestructureerde tekst

te detecteren en te redigeren.¹⁸ Het ondersteunt van nature meerdere talen, waaronder Nederlands, wat een belangrijk voordeel is.¹⁸

DataFog profileert zich als een extreem snel (lightning-fast) alternatief binnen het Python-ecosysteem.³ Evaluaties tonen aan dat DataFog ordegroottes sneller kan zijn dan de Presidio Analyzer.³ Echter, de trade-off voor deze snelheid is een aanzienlijk hoger aantal fout-positieven; het kan willekeurige nummerreeksen onterecht als IP-adressen identificeren, wat achteraf programmatische validatie (bijvoorbeeld een probe in Python om te testen of de string een geldig IP-object kan vormen via de `ipaddress` library) vereist om te voorkomen dat betekenisvolle data onnodig wordt geredigeerd.³

Het gebruik van LLMs vereist echter aanzienlijke rekenkracht (VRAM). Daarom ligt de focus steeds meer op efficiënte Small Language Models, wat ons direct bij de onderliggende hardware-abstractielaag brengt.

Lokale Inferentie-Infrastructuur: De Sleutelrol van Ollama

Het opzetten van private AI-infrastructuur vereist een robuuste, gestandaardiseerde inferentie-engine. Ollama heeft zich in ijltempo gevestigd als de de facto standaard voor het lokaal hosten en draaien van LLMs en SLMs op on-premise hardware.⁵ Men kan de functie van Ollama conceptueel vergelijken met wat Docker deed voor softwareapplicaties: het pakt de complexe afhankelijkheden van modellen, gewichten (weights), quantisaties (zoals GGUF-bestanden), CUDA-drivers, tensor-allocaties en geheugenbeheer naadloos in.⁵ Met een simpel commando zoals `ollama run llama3.1` haalt de software het gecomprimeerde (bijv. 4-bit quantized) model van 4.7 GB op en stelt het beschikbaar op een interne poort.⁴

Het meest cruciale architecturale voordeel van Ollama is de ingebouwde REST API, die volledig compatibel is met het OpenAI API ecosysteem.⁵ Dit stelt organisaties in staat om bestaande applicaties en integraties die oorspronkelijk zijn ontwikkeld voor de cloudgebaseerde OpenAI-diensten, naadloos lokaal te migreren door simpelweg de `base_url` parameter aan te passen naar `http://localhost:11434/v1` en een dummy API-key op te geven.²² Geen enkele byte aan gevoelige data verlaat het beveiligde, interne netwerk.⁵

Domeinspecifieke en Nederlandstalige Modellen

Voor inzet in het Nederlandse hoger onderwijs is de beschikbaarheid van taalmodellen die de grammaticale en culturele nuances van het Nederlands tot in de perfectie beheersen van essentieel belang. Grote, meertalige modellen zoals Llama 3.1, Mixtral of Gemma presteren adequaat, maar slaan de plank soms mis bij complexe Nederlandse zinsconstructies of landspecifieke PII-formaten.

Om dit hiaat op te vullen, heeft de gemeenschap—met voorop pioniers zoals Bram Vanroy—aanzienlijke inspanningen geleverd om modellen specifiek voor de Nederlandse

context te trainen. Modellen zoals **GEITje** (bijvoorbeeld de bramvanroy/geitje-7b-ultra variant, welke is gealigneerd via AI-feedback) en **Fietje** zijn doorontwikkelingen en fine-tunes van Llama- en Mistral-architecturen, getraind op omvangrijke, web-gecrawelde Nederlandse datasets en gespecialiseerde Nederlandse instructiesets.⁴¹ De instructiesets zijn specifiek vervaardigd om het redeneringsvermogen in het Nederlands te verhogen.⁴²

Dergelijke modellen vertonen een fundamenteel dieper begrip van Nederlandse tekststructuren. Dit correleert direct met een beduidend hogere nauwkeurigheid en lagere false-negative ratio's bij de detectie van PII in Nederlandstalige documenten of chatlogs. Wanneer men een SLM via Ollama inzet als een PII-filter, instrueert men het model via de system prompt. Een optimaal prompt-ontwerp vereist van het model dat het alle PII vervangt door unieke placeholders in het formaat ```, en strikt antwoordt met een JSON-object dat twee velden bevat: text (de geanonimiseerde tekst) en mapping (een woordenboek met de originele waarden gekoppeld aan de placeholders).¹⁶ Een model als GEITje begrijpt complexe Nederlandse context ("De heer Van den Broek stuurde een e-mail...") veel accurater dan een generiek model, waardoor pseudonimisering uiterst precies kan worden uitgevoerd.¹⁶

Voor de analyse van uiterst ongestructureerde data, zoals IT-serverlogs binnen de universitaire infrastructuur, bewijst het open-source project **PII Guard** dat compacte modellen (zoals gemma:3b), draaiend via de Ollama runtime, uiterst capabel zijn. Ze identificeren PII op basis van semantische context in situaties waar regex hopeloos faalt, en detecteren niet alleen namen, maar ook gevoelige categorieën volgens Artikel 9 van de AVG, zoals politieke opvattingen of gezondheidsgegevens, die indirect in tekst besloten liggen.² De lage VRAM footprint van SLMs maakt het mogelijk om op één GPU-server meerdere modellen parallel te hosten: één robuust model voor het genereren van output voor gebruikers, en een cluster van efficiënte SLMs die exclusief als in-stream PII-filters dienen op de achtergrond.

Orkestratie en Gebruikersinteractie via Open WebUI

Om de lokaal draaiende LLM-infrastructuur toegankelijk en gebruiksvriendelijk te maken voor collega's, docenten en onderzoekers zonder uitgebreide technische achtergrond, is een intuïtieve interface vereist. **Open WebUI** (voorheen bekend als Ollama WebUI) heeft de markt veroverd niet simpelweg als een ChatGPT-kloon, maar als een uitgebreid, modulaire platform voor AI-orkestratie, speciaal gebouwd voor lokale infrastructuur.⁴⁶

Wat Open WebUI uitzonderlijk krachtig maakt voor deployment in academische bedrijfsomgevingen is de diepgaande "Extensibility" (uitbreidbaarheid) architectuur. Deze stelt beheerders in staat om het platform aan te passen met Python, HTTP-aanroepen of community-plugins, waardoor modellen actuele data kunnen opvragen, policy-regels kunnen afdwingen en gekoppeld kunnen worden aan externe diensten zonder de kernconfiguratie van het model aan te tasten.⁴⁷

Voor privacy en databescherming biedt Open WebUI mechanismen om de communicatiestroom tussen de eindgebruiker en het onderliggende LLM onzichtbaar te

intercepteren en te modificeren.⁴⁹ Dit wordt gerealiseerd via twee primaire, fundamenteel verschillende concepten: Filter Functions en Pipelines.⁴⁹

Filter Functions: In-Process Interceptie

Filter Functions zijn compacte Python-scripts die direct in het RAM van het Open WebUI-serverproces (in-process) worden uitgevoerd.⁴⁹ Deze functies fungeren als middleware die elke inkomende user prompt (de inlet hook) en elke door het LLM gegenereerde uitgaande respons (de outlet hook) passeert en kan muteren.⁴⁹ De creatie en installatie van deze filters is strikt beperkt tot administrators om veiligheidsredenen, aangezien malafide Python-code toegang zou kunnen krijgen tot het interne bestandssysteem.⁵²

Een specifiek **PII Redaction Filter** werkt als volgt:

1. Een student of onderzoeker typt een vraag in de interface die een dataset met namen of telefoonnummers bevat.
2. Voordat de request de server verlaat richting Ollama, onderschept de inlet functie het bericht. De Python-code in de filter gebruikt reguliere expressies of roept een externe microservice (zoals Presidio of GLiNER) aan om de tekst razendsnel te zuiveren.⁴⁹
3. De gedetecteerde PII wordt vervangen door gemaskeerde entiteiten. Pas deze gemuteerde, schone tekst wordt als input aan het LLM geleverd.⁴⁹
4. Indien bidirectionele pseudonimisering is vereist, slaat de inlet functie de mapping lokaal op. Zodra het LLM het antwoord retourneert (bijvoorbeeld een samenvatting van het document), onderschept de outlet functie deze respons, zoekt naar de placeholders, en plaatst de originele persoonsgegevens terug in de tekst voordat deze op de interface van de gebruiker verschijnt.⁴⁹

Beheerders kunnen in Open WebUI "Always-On Filters" configureren (zonder een toggle-eigenschap in de UI). Dit betekent dat het filter onzichtbaar en permanent op de achtergrond draait voor elke gebruiker, ongeacht welk LLM-model zij hebben geselecteerd, waardoor organisatiebreed compliance beleid dwingend wordt opgelegd en prompt-injections proactief worden geblokkeerd.⁵²

Pipelines: Geïsoleerde Verwerkingsomgevingen

Hoewel Filter Functions in-process draaien en daardoor minimale netwerklentie introduceren, zijn ze in hun functionaliteit beperkt tot standaard Python-pakketten die binnen de Open WebUI-container beschikbaar zijn.⁴⁹ Voor zware, resource-intensieve bibliotheken zoals spacy, torch of het volledige presidio-analyzer pakket, raadt de architectuur het gebruik van **Pipelines** aan.⁴⁹

Een Pipeline wordt gehost als een afzonderlijke worker in een geïsoleerde Docker-container.⁴⁹ Het presenteert zich aan Open WebUI als een standaard, OpenAI-compatibel model of endpoint.⁴⁷ Dit is de technologisch superieure methode voor het offloaden van complexe of gevoelige verwerkingen. Een uitstekend praktijkvoorbeeld uit de community is de

presidio_filter_pipeline.py.⁵⁴ Binnen deze Pipeline-container draait het complete Presidio-framework in het geheugen.⁵⁴ Wanneer een gebruiker data verstuurt, fungeert de Pipeline als een proxy. Het analyseert en anonimiseert de tekst uitgebreid met Presidio, roept vervolgens asynchroon Ollama aan voor het LLM-werk, en sluist het antwoord terug naar de web UI.⁴⁹ Deze architectuur zorgt ervoor dat de hoofdapplicatie (Open WebUI) schoon, stabiel en snel blijft, terwijl de zware detectietaken profiteren van GPU-versnelling in hun eigen container.⁴⁹

Verder wordt bediscussieerd om ook databasetransacties in Open WebUI (het opslaan en ophalen van eerdere chatgeschiedenis) door een filter te halen, zodat de op de schijf opgeslagen data (at rest) blijvend is geanonimiseerd. Hoewel nog niet native perfect geïmplementeerd voor database I/O, biedt een asynchrone pipeline-benadering gecombineerd met Presidio hier een uiterst werkbare en veilige oplossing.⁵⁵

Beveiliging van Retrieval-Augmented Generation (RAG) Systemen

Naast directe chatinteracties, ligt de werkelijke kracht van AI in het onderwijs in het analyseren van de eigen, interne kennisbank van de instelling. Dit wordt bereikt via Retrieval-Augmented Generation (RAG). In een typische RAG-pijplijn worden interne documenten (HR-richtlijnen, scripties, onderzoekspublicaties) ingeladen, in kleinere semantische brokken opgedeeld (chunking), omgezet in wiskundige vectoren (embeddings), en opgeslagen in een schaalbare vector-database, zoals Elasticsearch, ChromaDB of Milvus.¹ Wanneer een gebruiker vervolgens een vraag stelt via Open WebUI, vergelijkt het systeem de inbedding van de vraag met de database, haalt de meest relevante tekstblokken (chunks) op, en voegt deze als contextuele feiten toe aan de prompt voor het taalmodel.⁵⁷

De integratie van RAG introduceert echter een colossaal en fundamenteel veiligheidsrisico op het gebied van PII.¹ Als ongeschoonde documenten worden ingevoerd, verblijft alle gevoelige data permanent en vrij opzoekbaar in de vector-database.¹ Tijdens het semantisch ophalen kan het systeem onbedoeld informatie van een andere student, of medisch gevoelige onderzoeksinformatie, extraheren en injecteren in het antwoord aan de huidige gebruiker, wat een catastrofaal datalek veroorzaakt.

Om RAG-systemen AVG-conform te maken, moet de anonimisering dwingend plaatsvinden voordat de data wordt geëmbend en opgeslagen in de vector-database.⁵⁷

LangChain en LlamaIndex Integraties

Populaire data-frameworks zoals LlamaIndex en LangChain, die veelvuldig worden ingezet om deze RAG-architecturen te bouwen, bieden robuuste, ingebouwde mechanismen voor data-sanering.¹

Binnen het **LangChain** ecosysteem, specifiek in integratie met LangSmith, kan de ontwikkelaar

gebruikmaken van de `create_anonymizer` functionaliteit.²² Hierbij kan men een lokale LLM via Ollama aanspreken.²² Door bijvoorbeeld een compact model via `http://127.0.0.1:1234/v1` aan te roepen met `temperature=0` (om deterministische, consistente resultaten te forceren), wordt de tekst geanalyseerd. Het LLM wordt geprompt met instructies zoals: "Vervang PII: *names*→[NAME], *emails*→[EMAIL]... Retourneer alleen de tekst."²² De gezuiverde tekst wordt vervolgens geëmbend, waardoor de resulterende trace en vectoropslag volstrekt geen PII bevatten.²² De trade-off is uiteraard extra latentie (ongeveer 100-500ms per RAG chunk) vanwege de LLM aanroepen.²²

LlamaIndex pakt dit structureel op via abstracties genaamd `Node Postprocessors`.⁵⁷ Tijdens de data-ingestie passeert elk tekstknooppunt (`Node`) de `postprocessor`. LlamaIndex biedt hier standaardcomponenten voor:

- De `NERPIINodePostprocessor` integreert direct een lokaal Hugging Face NER-model.⁵⁷ Wanneer dit over een tekst zoals "Mijn creditcard 1111-0000-1111 is gestuurd naar 123 Any Street" loopt, retourneert het model de geredigeerde tekst en, uiterst belangrijk, slaat het de mapping op in een afgeschermd metadataveld `__pii_node_info__` van de `Node`.⁶⁰ Cruciaal is dat deze specifieke `node_info` parameter *nooit* aan het taalmodel of het embedding-model wordt meegeleverd, maar uitsluitend lokaal in het scriptgeheugen of in een aparte key-value store blijft.⁶⁰
- De `PIINodePostprocessor` is de equivalente implementatie die een lokaal LLM via Ollama inzet voor hetzelfde proces, met de instructie om placeholders zoals `` in te vullen en een mapping-dictionary te genereren.⁵⁷

Door deze strikte architectonische scheiding te handhaven, is de vector-database (het lange-termijn geheugen van het RAG systeem) en het latere generatieve taalmodel immuun voor PII-besmetting, terwijl het platform desondanks in staat is om antwoorden veilig te deanonymiseren voordat ze via de interface worden uitgeleverd aan een hiertoe geautoriseerde docent of onderzoeker.

Praktische Gids: Implementatie op een GPU-Infrastructuur

Om de behandelde theorie om te zetten naar een robuuste, deployable architectuur op een server uitgerust met GPU's (bijvoorbeeld ter ondersteuning van een instellingsbrede pilot), presenteren we hieronder een stapsgewijze, technische implementatiegids. De focus ligt op robuustheid, containerisatie en specificatie voor de Nederlandstalige context.

1. Systeemarchitectuur en Docker Compose Orkestratie

De basis van de infrastructuur is een georkestreerde Linux-omgeving gestuurd door Docker Compose.³⁹ Door te kiezen voor containerisatie garandeert men reproduceerbaarheid en isolatie over verschillende hardwareomgevingen, wat de schaalbaarheid en stabiliteit

waarborgt.³⁹ De architectuur omvat de Ollama LLM-engine, de Open WebUI server, en een lokaal gehoste Presidio Analyzer en Anonymizer via Flask.

Een fundamentele eis is dat de Ollama-container directe, fysieke toegang krijgt tot de NVIDIA GPU's om tensor operaties af te handelen.⁶² Dit wordt geregeld via Docker's `--gpus all` of via de `deploy resources` configuratie in `compose`.⁶²

Een representatief en direct inzetbaar `docker-compose.yml` script voor deze complete stack is als volgt gestructureerd:

YAML

```
version: "3.8"
services:
  # De Ollama Inferentie Engine
  ollama:
    image: ollama/ollama:latest
    container_name: ollama-gpu
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]
    restart: unless-stopped
    healthcheck:
      test:
        interval: 10s
        timeout: 10s
        retries: 30
        start_period: 60s

  # De Open WebUI Interface en Orkestrator
  open-webui:
    image: ghcr.io/open-webui/open-webui:main
    container_name: open-webui
```

```

ports:
  - "3000:8080"
environment:
  - OLLAMA_BASE_URL=http://ollama:11434
volumes:
  - open-webui_data:/app/backend/data
depends_on:
  ollama:
    condition: service_healthy
restart: unless-stopped

# Microsoft Presidio Analyzer Service
presidio-analyzer:
  image: mcr.microsoft.com/presidio-analyzer:latest
  container_name: presidio-analyzer
  environment:
    - PORT=3000
  ports:
    - "5002:3000"
  restart: unless-stopped

# Microsoft Presidio Anonymizer Service
presidio-anonymizer:
  image: mcr.microsoft.com/presidio-anonymizer:latest
  container_name: presidio-anonymizer
  environment:
    - PORT=3000
  ports:
    - "5001:3000"
  restart: unless-stopped

volumes:
  ollama_data:
  open-webui_data:

```

Via de uitvoering van het commando `docker compose up -d` spint de volledige infrastructuur op in een afgeschermd virtueel Docker-netwerk.⁴⁵ De diensten communiceren uitsluitend intern. De host-server levert uitsluitend de front-end UI van Open WebUI op poort 3000.⁶³

Na initialisatie kunnen de benodigde Grote Taalmodellen asynchroon in de achtergrond worden gedownload via een post-startup commando in de terminal (bijvoorbeeld: `docker exec -it ollama-gpu ollama pull llama3.2` of specifiek voor de Nederlandstalige context: `ollama run bramvanroy/geitje-7b-ultra:Q3_K_M`).⁴ Geitje-7B vereist in dit gequantiseerde format (Q3_K_M)

zeer bescheiden VRAM en functioneert exceptioneel goed voor Nederlandstalige logica en chatfunctionaliteiten.⁴¹

2. Ontwikkeling en Python Integratie van Nederlandse PII Recognizers

Voor de integratie met Microsoft Presidio is vastgesteld dat lokalisatie noodzakelijk is.²⁹ In plaats van het inzetten van de standaard Presidio Docker image, is de best-practice voor onderzoeksinstellingen om een eigen Dockerfile te bouwen, gebaseerd op het Presidio image, waarin geavanceerde Python-scripts worden geïnjecteerd. De twee meest uitdagende, lokaal relevante PII entiteiten zijn Nederlandse telefoonnummers en het Burgerservicenummer (BSN).⁶⁴

Regex Optimalisatie voor het Nederlandse Telefoonnummer

Een ogenschijnlijk simpel probleem—de herkenning van een telefoonnummer—stuit in de praktijk op aanzienlijke complexiteit. Nederlandse nummers verschijnen in talloze formattingen: 0612345678, 06-12345678, +31612345678, 00316-12345678, of vaste nummers zoals 0345-123456.⁶⁵ Een naïeve reguliere expressie die louter naar cijfers en koppelteken zoekt, resulteert in onacceptabele fout-positieven, zoals het maskeren van postcodes gecombineerd met huisnummers, of reeksen van datums (06-12-3456).⁶⁵

De geoptimaliseerde en meest robuuste regex patroon voor de Nederlandse context is als volgt geconstrueerd: `^(?:(?:\+|00)31|0)(?:\d-?\d{8}|\d\d-\d{7})|0\d{3}-\d{6})$`.⁶⁵

Deze expressie is opgedeeld in non-capture groups (`(?:...)`).⁶⁵ Het zoekt specifiek naar landcodes (+31 of 0031) of een lokale nul (0), gevolgd door exact negen resterende cijfers voor mobiele nummers (met tolerante acceptatie van één koppelteken), en controleert specifieke verhoudingen (drie-om-zes cijfers) voor vaste geografische netlijnen.⁶⁵

Om deze krachtige regex in Presidio in te bouwen, dient men een ad-hoc of gecodeerde PatternRecognizer te specificeren.²⁶ Cruciaal is om hierbij contextwoorden te definiëren, waardoor de AnalyzerEngine de betrouwbaarheidsscore exponentieel kan verhogen als het telefoonnummer voorkomt nabij specifieke linguïstische aanduidingen.¹⁷

Python

```
from presidio_analyzer import PatternRecognizer, Pattern
```

```
# Definitie van het Regex patroon
```

```
nl_phone_pattern = Pattern(
```

```
    name="Dutch_Phone_Regex",
```

```
    regex=r"^(?:(?:\+|00)31|0)(?:\d-?\d{8}|\d\d-\d{7})|0\d{3}-\d{6})",
```

```

    score=0.4 # Lage basis-score om false-positives bij toevalige getallen te voorkomen
)

# Integratie in de PatternRecognizer met Linguïstische Context
nl_phone_recognizer = PatternRecognizer(
    supported_entity="NL_PHONE_NUMBER",
    patterns=[nl_phone_pattern],
    context=["telefoon", "tel", "mobiel", "06", "nummer", "bereikbaar", "bellen"],
    supported_language="nl"
)

# De registry load() logica voegt de recognizer vervolgens toe aan de Engine

```

Wanneer de Analyzer over een tekst parseert en de regex een match rapporteert, is de beginscore slechts 0.4. Detecteert de engine echter binnen de lemma-context het woord "mobiel", dan activeert de score_context_improvement parameter, waardoor de betrouwbaarheid naar bijvoorbeeld 0.85 wordt geduwd, waarna effectieve redactie volgt.¹⁷

Cryptografische Validatie: De Elfproef voor het BSN

Voor het Burgerservicenummer (BSN), net als voor studentnummers die gekoppeld zijn aan landelijke DUO-protocollen, is een simpele string van 8 of 9 cijfers volstrekt ontoereikend voor identificatie.¹⁷ Dit patroon zou zonder twijfel willekeurige onderzoeks ID's of data-arrays maskeren. Het Nederlandse BSN is wiskundig beschermd middels de Elfproef (een checksum algoritme).¹⁷

Om te voorkomen dat Presidio buitensporig PII detecteert (false positives), moet de herkenner worden geschreven als een volledige, overervende Python class (EntityRecognizer) die het wiskundige validatie mechanisme afdwingt.¹⁷

Wanneer het patroon negen cijfers detecteert, roept de Analyzer een interne validatiefunctie aan op de reeks. De algoritme-logica is: $1 \times c_1 + 2 \times c_2 + \dots + 9 \times c_9 \equiv 0 \pmod{11}$

waarbij c_n het n-de cijfer (van achter naar voren geteld) is, met een specifieke aftrek op het laatste cijfer. Als de wiskundige sommatie na de modulo bewerking exact 0 oplevert, verandert de in Presidio gedefinieerde validation_result vlag naar True.¹⁷ Alleen op dat exacte moment stijgt de betrouwbaarheidsscore richting de 1.0 (100%), en wordt de reeks gemarkeerd als legitiem BSN, gereed voor anonymisatie. Het is deze diepgaande abstractielaag die Presidio geschikt maakt voor productie-kritische inzet in het onderwijs.

3. Configuratie van de Open WebUI Pipeline

Nadat de presidio-analyzer (voorzien van de Python custom recognizers) en de presidio-anonymizer draaien als robuuste microservices via de lokale poorten (resp. 3000 mapping naar 5002 en 5001), dient het proces geïntegreerd te worden in de gebruikersinterface.²⁸ Binnen de Open WebUI Workspace (Settings -> Functions / Pipelines)

implementeert de beheerder een Filter Function of Pipeline-script.⁵²

Een typerend filter python script, gebaseerd op de Open WebUI documentatie ⁵⁰, fungeert als volgt:

1. De inlet hook intercepteert de binnenkomende chat array.
2. Het script construeert een JSON payload waarin de tekst wordt gedefinieerd, evenals de gevraagde taal (nl) en de beoogde entiteiten.²⁵
3. Via een synchrone Python HTTP requests.post call wordt de data doorgestuurd naar het adres `http://presidio-analyzer:3000/analyze`.²⁵
4. Het ontvangen antwoord (de RecognizerResult array) wordt direct samengevoegd met de originele tekst en opnieuw als HTTP POST verzonden naar `http://presidio-anonymizer:3000/anonymize`, vergezeld van specifieke redactie regels (zoals maskering van telefoonnummers met sterretjes, of het genereren van een hash voor het BSN).²⁸
5. De uiteindelijke respons—de structureel geanonimiseerde tekst—vervangt in-memory het oorspronkelijke chat bericht, waarna de veilige payload definitief wordt afgevuurd op het Ollama LLM endpoint.⁴⁹

Dit gehele, complexe technische ballet voltrekt zich asynchroon binnen fracties van seconden, en is volkomen onzichtbaar voor de docent of student die de interface gebruikt.⁴⁹

Kwantitatieve Vergelijking en Architectuurkeuze

Om bestuurders en IT-architecten in het hoger onderwijs te faciliteren bij het maken van beleidskeuzes aangaande de toe te passen technologieën, volgt hieronder een kwantitatieve en kwalitatieve, gestructureerde vergelijking van de gepresenteerde extractiemethoden:

Methodologie / Tool	Operatie Latentie (Snelheid)	GPU VRAM Vereisten	Complexiteit Aanpasbaarheid	Semantisch & Contextueel Begrip	Kenmerkende Fout-Positief Ratio (FPR)	F1-Score / Recall Karakteristiek
Microsoft Presidio (Default / Regex)	Extreem Snel (< 50ms) ³	Nul (Puur CPU)	Hoog (Via Python Regex classes) ²⁷	Beperkt (Louter lokaal contextwoord herkenning) ²³	Hoog bij rommelige tekst, Extreem Laag bij formele structure	Hoge Precision, Lage Recall

					n (e.g., checksu ms) ¹⁷	
Microsoft Presidio met spaCy NLP	Snel (~100-200ms)	Zeet Laag (CPU / Lichte GPU RAM)	Zeet Laag (Vereist zware annotatie en model hertrainin g) ¹⁴	Gemiddel d (Afhankel ijk van NLP window) ²³	Gemiddel d	Gemiddel de Precision / Recall
GLiNER (Zero-Shot Transformers)	Gemiddel d (~200-500ms) ²¹	Gemiddel d (Optimalisatie via ONNX en FP16) ²¹	Extreem Hoog (Aanpasbaar middels dynamische prompts op runtime) ²¹	Hoog (Bidirectionele encoder architectuur) ²¹	Laag (Dankzij geavanceerde fine-tuning)	Hoge F1-score (~81%), Uiterst hoge Recall ³³
SLMs via Ollama (bijv. GEITje / Llama)	Traag (> 1000ms+)	Zeet Hoog (Specifieke enterprise GPU's vereist voor concurrentie)	Uitmuntend (Volledig taalkundig en contextueel begrip via prompts) ¹⁶	Maximaal (Semantische redenering) ²	Zeet Laag (echter aanzienlijk risico op onvoorspelbare model-hallucinaties)	Variabel afhankelijk van Model parameterisatie

Uit deze objectieve vergelijking kristalliseert zich een duidelijke synthese uit: een hybride architectuur geniet onomstotelijk de absolute voorkeur voor enterprise-grade omgevingen.³⁸ Rigide, formeel gestructureerde identifiers (waaronder het BSN met 11-proef, IBAN-nummers, IPv4/IPv6-adressen en strak geformatteerde E-mailadressen) dienen onherroepelijk te worden onderschept door Microsoft Presidio's Analyzer. Dit waarborgt snelheid en computationele efficiëntie voor de bulk van de PII.³ Complexe, ongestructureerde, contextueel diep genestelde en onvoorspelbare persoonsgegevens (zoals onverwachte roepnamen die verborgen zitten in

lange, getypte interviewtranscripties of specifieke locatieduidingen in een beleidsdocument) kunnen het meest robuust worden gedetecteerd door het integreren van een tussenliggende GLiNER-module, of door ze batch-gewijs te laten verwerken door een gespecialiseerd Small Language Model via Ollama en projecten als AnonDocs of PII Guard.²¹

Conclusies en Strategische Aanbevelingen

De doordachte, verantwoorde integratie van geavanceerde AI-technologie in het hoger onderwijs vereist een uiterst delicaat, continu evenwicht tussen het aanjagen van academische en operationele innovatie enerzijds, en fundamentele, compromisloze privacybescherming anderzijds. Het lijdzaam afhankelijk blijven van gesloten, externe cloud-modellen (vendor lock-in) en het ongefilterd exporteren van onversleutelde, privacygevoelige brondata vormen onaanvaardbare, onrechtmatige risico's met betrekking tot datasoevereiniteit en de handhaving van de Europese Algemene Verordening Gegevensbescherming (AVG). Het in eigen beheer opzetten, finetunen en orkestreren van een lokaal gehoste, on-premise LLM-infrastructuur op interne GPU-servers (naar het voorbeeld van SURF-pilots en de AI-Hub principes) is hiertoe de enige verdedigbare en toekomstbestendige architectonische oplossing.

Op basis van de technologische evaluaties, de software-analyses en de praktische systeemarchitectuur uiteengezet in dit rapport, worden de volgende vijf overkoepelende, strategische aanbevelingen geformuleerd voor instituten:

1. **Standaardiseer de Fundamenten op Ollama en Open WebUI:** Implementeer en omarm Ollama als de gestandaardiseerde, wendbare inferentiemotor voor modellen, tezamen met Open WebUI als het centrale, schaalbare beheer- en interactieplatform. Deze open-source stack biedt onovertroffen flexibiliteit, fijnmazige beheerderscontrole en naadloze backward compatibility met de industriestandaard OpenAI API-specificaties, hetgeen cruciaal is voor frictieloze adoptie door interne development teams en academische stafleden die reeds geacclimatiseerd zijn aan commerciële tools.
2. **Hanteer een Hybride, Meerlaagse Anonimiseringsstrategie:** Vertrouw nadrukkelijk niet op één enkele methodologie voor de identificatie van PII. Combineer de onverbiddelijke snelheid en computationele efficiëntie van deterministische patroonherkenning (via Microsoft Presidio, uitgerust met agressief geparameteriseerde Nederlandse regex-herkenners en cryptografische elfproef-validaties voor BSN's) met de diepe, associatieve contextuele intelligentie van neurale entiteitsmodellen (waarbij de toepassing van gespecialiseerde GLiNER varianten of lokaal via Ollama draaiende SLMs zoals GEITje sterk wordt aanbevolen).
3. **Afdwinging via Onzichtbare In-Stream Filtering:** Maak voor productie-omgevingen dwingend en intensief gebruik van de Extensibility-features van Open WebUI. Configureer Always-On Filter Functions in Python, of bouw zware verwerkingen uit tot geïsoleerde Pipelines, die als ijzeren middleware opereren. Dit garandeert dat werkelijk alle inkomende user prompts en uitgaande model-responses verplicht, en buiten de controle van de individuele gebruiker om, een rigoureuze detectie- en anonimiseringsfase doorlopen. Dit systeem design verankert het concept van "Privacy by

Design" onlosmakelijk in de architectuur van het platform.

4. **Harde Isolatie in Data Ingestie en RAG-pijplijnen:** Bij de ontwikkeling en bouw van interne kennisbank-systemen (Retrieval-Augmented Generation, oftewel het chatten met eigen documenten), dient de anonimisering dwingend stroomopwaarts te worden toegepast, expliciet *voorafgaand* aan de creatie van vector embeddings en indexering van de tekst. Het systematisch inzetten van gespecialiseerde architecturale blokken zoals Node Postprocessors in LlamaIndex of de ingebouwde anonymizer-haken in LangChain verzekert dat persistente vector-databases te allen tijde gevrijwaard blijven van privacygevoelige, te herleiden persoonsgegevens. Dit vormt de ultieme implementatie van structurele dataminimalisatie voor AI.
5. **Focus op Doorontwikkelde Nederlandse Lokalisatie:** De waarde van een AI-systeem valt of staat met contextueel begrip. Prioriteer structureel de inzet, acquisitie en ondersteuning van specifieke taalmodellen en NLP-frameworks die niet slechts rudimentair meertalig zijn, maar diepgaand getraind, geoptimaliseerd of aligned zijn voor de nuances, syntaxis en culturele context van de Nederlandse taal. Investerings van resources in het configureren of finetunen van modellen en herkenkers naar academische, Nederlandstalige standaarden tenderen vrijwel direct naar een onmetelijk beter rendement. Zij resulteren proefondervindelijk in aanzienlijk lagere fout-positief ratio's, verhoogde utiliteit van de geanonimiseerde data en ultiem in een massale, veilige en vertrouwenswekkende acceptatie door eindgebruikers in de organisatie.

Geciteerd werk

1. PII Detector for RAG: Presidio Masking Guide | LlamaIndex, geopend op april 9, 2026, <https://www.llamaindex.ai/blog/pii-detector-hacking-privacy-in-rag>
2. I have created LLM based PII detection project : r/SideProject - Reddit, geopend op april 9, 2026, https://www.reddit.com/r/SideProject/comments/1kff3eo/i_have_created_llm_based_pii_detection_project/
3. Detecting PII with Python - Mostafa Moradian, geopend op april 9, 2026, <https://mostafa.dev/detecting-pii-with-python-4c0da7b3b5bd>
4. Using a small local LLM (Llama 3.1 8B) to anonymize data for a remote big LLM (ChatGPT/Claude Sonnet) | by Sacha Storz | Medium, geopend op april 9, 2026, <https://medium.com/@scmstorz/using-a-small-local-llm-llama-3-1-1d13223b2bbe>
5. How to Protect Sensitive Data by Running LLMs Locally with Ollama - freeCodeCamp, geopend op april 9, 2026, <https://www.freecodecamp.org/news/protect-sensitive-data-with-local-llms/>
6. Privacy explained | SURF.nl, geopend op april 9, 2026, <https://www.surf.nl/en/themes/privacy/privacy-explained>
7. Towards transparent, fair and verifiable use of AI | SURF.nl, geopend op april 9, 2026, <https://www.surf.nl/en/news/towards-transparent-fair-and-verifiable-use-of-ai>
8. DPIA on EduGenAI | SURF, geopend op april 9, 2026, <https://www.surf.nl/files/2025-09/250808-dpia-on-edugenai-for-surf.pdf>

9. AI hub | SURF.nl, geopend op april 9, 2026,
<https://www.surf.nl/en/themes/artificial-intelligence/projects-and-collaborations/ai-hub>
10. F.A.Q. - SURF User Knowledge Base, geopend op april 9, 2026,
<https://servicedesk.surf.nl/wiki/spaces/WIKI/pages/222464716/F.A.Q.>
11. Reports and publications | AI - SURF, geopend op april 9, 2026,
<https://www.surf.nl/en/themes/artificial-intelligence/reports-and-publications>
12. Anonymize your PII data before sending it to an LLM - mstack, geopend op april 9, 2026, <https://mstack.nl/blogs/anonymize-pii-llm/>
13. De beste tools voor dataanonymisering en technieken van de volgende generatie | Syntho, geopend op april 9, 2026,
<https://www.syntho.ai/nl/the-best-data-anonymization-tools-next-gen-techniques/>
14. PII anonymization made easy by Presidio | by Lingzhen Chen | TDS Archive - Medium, geopend op april 9, 2026,
<https://medium.com/data-science/building-a-customized-pii-anonymizer-with-microsoft-presidio-b5c2ddfe523b>
15. End-to-end pseudonymization of fine-tuned clinical BERT models - PMC - NIH, geopend op april 9, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC11197357/>
16. MTBS/anonymizer/system - Ollama, geopend op april 9, 2026,
<https://ollama.com/MTBS/anonymizer/blobs/75ef1625402c>
17. Microsoft Presidio: an engineer's introduction to PII detection and de-identification - Medium, geopend op april 9, 2026,
<https://medium.com/neural-engineer/microsoft-presidio-an-engineers-introduction-to-pii-detection-and-de-identification-6a7c3fed6e50>
18. How Good Are Open-Source LLM-Based De-identification Tools in a Medical Context? | by Muhammet SANTAS | John Snow Labs | Medium, geopend op april 9, 2026,
<https://medium.com/john-snow-labs/how-good-are-open-source-llm-based-de-identification-tools-in-a-medical-context-6600ddac6a0f>
19. PII entities supported by Presidio, geopend op april 9, 2026,
https://microsoft.github.io/presidio/supported_entities/
20. How to deal with different languages · Issue #373 · microsoft/presidio - GitHub, geopend op april 9, 2026, <https://github.com/microsoft/presidio/issues/373>
21. knowledgator/gliner-pii-base-v1.0 - Hugging Face, geopend op april 9, 2026,
<https://huggingface.co/knowledgator/gliner-pii-base-v1.0>
22. Using Local LLM and LangChain Anonymizer for PII Anonymization, geopend op april 9, 2026,
<https://support.langchain.com/articles/7720970706-using-local-llm-and-langchain-anonymizer-for-pii-anonymization?threadId=99c8f147-4819-410d-a157-8cfd7f07af46>
23. Presidio in Action: Detecting and Securing PII in Text | by Lakmina Pramodya Gamage, geopend op april 9, 2026,
<https://blog.stackademic.com/presidio-in-action-detecting-and-securing-pii-in-text-451711e3c544>

24. GitHub - microsoft/presidio: An open-source framework for detecting, redacting, masking, and anonymizing sensitive data (PII) across text, images, and structured data. Supports NLP, pattern matching, and customizable pipelines., geopend op april 9, 2026, <https://github.com/microsoft/presidio>
25. Presidio Analyzer, geopend op april 9, 2026, <https://microsoft.github.io/presidio/analyzer/>
26. presidio/presidio-analyzer/tests/test_recognizer_registry.py at main - GitHub, geopend op april 9, 2026, https://github.com/microsoft/presidio/blob/main/presidio-analyzer/tests/test_recognizer_registry.py
27. Tutorial - Microsoft Presidio - Microsoft Open Source, geopend op april 9, 2026, https://microsoft.github.io/presidio/analyzer/adding_recognizers/
28. Using Presidio in Docker, geopend op april 9, 2026, <https://microsoft.github.io/presidio/samples/docker/>
29. Multi-language support - Microsoft Presidio, geopend op april 9, 2026, <https://microsoft.github.io/presidio/analyzer/languages/>
30. Flair recognizer - Microsoft Presidio, geopend op april 9, 2026, https://microsoft.github.io/presidio/samples/python/flair_recognizer/
31. Add Dutch Phone Number · microsoft presidio · Discussion #1562 - GitHub, geopend op april 9, 2026, <https://github.com/microsoft/presidio/discussions/1562>
32. regex defined in ad_hoc_recognizers is always case-insensitive? · Issue #1603 · microsoft/presidio - GitHub, geopend op april 9, 2026, <https://github.com/microsoft/presidio/issues/1603>
33. Comparing Best NER Models For PII Identification - Protecto AI, geopend op april 9, 2026, <https://www.protecto.ai/blog/best-ner-models-for-pii-identification/>
34. How Good Are Open-Source LLM-Based De-identification Tools in a Medical Context?, geopend op april 9, 2026, <https://www.johnsnowlabs.com/how-good-are-open-source-llm-based-de-identification-tools-in-a-medical-context/>
35. Detect and obfuscate PII - Docling, geopend op april 9, 2026, https://docling-project.github.io/docling/examples/pii_obfuscate/
36. The Next Generation of Privacy: Using Docling & GLiNER's Advanced NER to Masterfully Detect and Obfuscate PII - Alain Airom (Ayrom), geopend op april 9, 2026, <https://alain-airom.medium.com/the-next-generation-of-privacy-using-docling-gliners-advanced-ner-to-masterfully-detect-and-obfuscate-pii-6bf8fe68b2ed>
37. AI-SmartTalk/AnonDocs · GitHub - GitHub, geopend op april 9, 2026, <https://github.com/AI-SmartTalk/AnonDocs>
38. PII Guard is an LLM-powered tool that detects and manages Personally Identifiable Information (PII) in logs — designed to support data privacy and GDPR compliance - GitHub, geopend op april 9, 2026, <https://github.com/rpgeeganage/pii-guard>
39. Deploying Ollama with Docker Compose: A Simple Guide to Local LLMs - Medium, geopend op april 9, 2026, <https://medium.com/@prasanta.mohanty/deploying-ollama-with-docker-compose>

[e-a-simple-guide-to-local-llms-610db2991581](#)

40. Ollama | Microsoft Learn, geopend op april 9, 2026,
<https://learn.microsoft.com/en-us/agent-framework/agents/providers/ollama>
41. bramvanroy/geitje-7b-ultra - Ollama, geopend op april 9, 2026,
<https://ollama.com/bramvanroy/geitje-7b-ultra>
42. BramVanroy/dutch-instruction-datasets - GitHub, geopend op april 9, 2026,
<https://github.com/bramvanroy/dutch-instruction-datasets>
43. bramvanroy - Ollama, geopend op april 9, 2026, <https://ollama.com/bramvanroy>
44. [2312.12852] Language Resources for Dutch Large Language Modelling - arXiv,
geopend op april 9, 2026, <https://arxiv.org/abs/2312.12852>
45. I built a self-hosted tool to detect PII (personally identifiable information) in logs
using AI (Node.js + Ollama + Elasticsearch) : r/devops - Reddit, geopend op april 9,
2026,
https://www.reddit.com/r/devops/comments/1khn7ac/i_built_a_selfhosted_tool_to_detect_pii/
46. How to Self-Host OpenWebUI: The Ultimate Guide to Your Own Secure AI Chat
Platform, geopend op april 9, 2026,
<https://www.youtube.com/watch?v=ndSwy3sIV4s>
47. Open WebUI: An Open, Extensible, and Usable Interface for AI Interaction - arXiv,
geopend op april 9, 2026, <https://arxiv.org/html/2510.02546v1>
48. open-webui/open-webui: User-friendly AI Interface (Supports Ollama, OpenAI
API ... - GitHub, geopend op april 9, 2026,
<https://github.com/open-webui/open-webui>
49. Extensibility - Open WebUI, geopend op april 9, 2026,
<https://docs.openwebui.com/features/extensibility/>
50. Open WebUI Filter Functions - Complete Tutorial - YouTube, geopend op april 9,
2026, https://www.youtube.com/watch?v=bm7oBOw_ss0
51. Tools Usage (Guide) · open-webui open-webui · Discussion #3134 - GitHub,
geopend op april 9, 2026,
<https://github.com/open-webui/open-webui/discussions/3134>
52. Filter Function: Modify Inputs and Outputs - Open WebUI, geopend op april 9,
2026, <https://docs.openwebui.com/features/extensibility/plugin/functions/filter/>
53. TTS Synthesis Filter Function & Community Filters on OpenWeb-UI Website
#13979 - GitHub, geopend op april 9, 2026,
<https://github.com/open-webui/open-webui/discussions/13979>
54. pipelines/examples/filters/presidio_filter_pipeline.py at main · open ..., geopend op
april 9, 2026,
https://github.com/open-webui/pipelines/blob/main/examples/filters/presidio_filter_pipeline.py
55. feat: Filter functions that are applied when writing to the database #11891 -
GitHub, geopend op april 9, 2026,
<https://github.com/open-webui/open-webui/discussions/11891>
56. Anonymize RAG data in IBM Granite and Ollama using HCP Vault - HashiCorp,
geopend op april 9, 2026,
<https://www.hashicorp.com/en/blog/anonymize-rag-data-in-ibm-granite-and-olla>

[ma-using-hcp-vault](#)

57. Protecting Sensitive and PII information in RAG with Elasticsearch and LlamaIndex, geopend op april 9, 2026,
<https://www.elastic.co/search-labs/blog/rag-security-masking-pii>
58. Handling PII data in LangChain, geopend op april 9, 2026,
<https://www.langchain.com/blog/handling-pii-data-in-langchain>
59. PII Masking | LlamaIndex Python Documentation - LlamaParse, geopend op april 9, 2026,
https://developers.llamaindex.ai/python/examples/node_postprocessor/pii/
60. PII Masking - LlamaIndex 0.6.18, geopend op april 9, 2026,
https://lama-index.readthedocs.io/zh/latest/examples/node_postprocessor/PII.html
61. Host Your Own AI Code Assistant with Docker, Ollama and Continue! - YouTube, geopend op april 9, 2026, <https://www.youtube.com/watch?v=OpmMeOmd0tA>
62. How to Use Ollama with Docker - OneUptime, geopend op april 9, 2026,
<https://oneuptime.com/blog/post/2026-01-27-ollama-docker/view>
63. presidio/docker-compose.yml at main - GitHub, geopend op april 9, 2026,
<https://github.com/microsoft/presidio/blob/main/docker-compose.yml>
64. Regular expression for Dutch phone number - regex - Stack Overflow, geopend op april 9, 2026,
<https://stackoverflow.com/questions/17949757/regular-expression-for-dutch-phone-number>
65. Optimize the Dutch telephone number RegEx pattern - Stack Overflow, geopend op april 9, 2026,
<https://stackoverflow.com/questions/69741785/optimize-the-dutch-telephone-number-regex-pattern>
66. Regular expression dutch mobile number or international number - Stack Overflow, geopend op april 9, 2026,
<https://stackoverflow.com/questions/43258607/regular-expression-dutch-mobile-number-or-international-number>
67. Models - Open WebUI, geopend op april 9, 2026,
<https://docs.openwebui.com/features/workspace/models/>
68. Open Source PHI De-Identification: A Technical Review | IntuitionLabs, geopend op april 9, 2026,
<https://intuitionlabs.ai/articles/open-source-phi-de-identification-tools>